

Laboratorio 7: NLP end-to-end y Embeddings

Anthony Lou Schwank · Carné 23410 · Octubre 2026

Resumen. Se construyó el pipeline texto→tokens→IDs→embeddings→clasificador sobre 30 M tokens de WikiText-103. Un skip-gram con negative sampling (SGNS) implementado desde cero en PyTorch alcanza 48.9 % en las analogías de Mikolov y $\rho = 0.674$ en WordSim-353, prácticamente igual que Word2Vec de gensim con la misma configuración (49.0 %, 0.669), y 16.6 puntos por debajo de GloVe (6 B tokens), brecha que se explica en buena parte por el tamaño del corpus. En AG News, los embeddings preentrenados con *fine-tuning* superan a los aleatorios (F1 0.914–0.918 vs. 0.906) y con 1 % de los datos los superan por 15–16 puntos, aunque con todos los datos el baseline TF-IDF (0.921) sigue siendo el mejor.

1. Investigación: embeddings en PyTorch y gensim

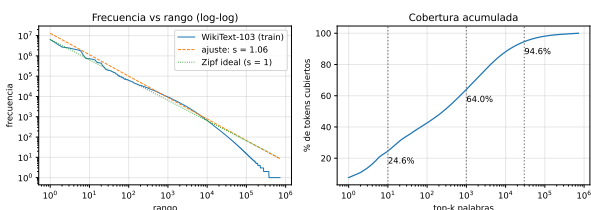
- `nn.Embedding(num_embeddings, embedding_dim, padding_idx, sparse)`: tabla entrenable $V \times d$; recibe IDs y devuelve sus filas. `padding_idx` fija una fila en cero sin gradiente (para <pad>); `sparse=True` produce gradientes sólo de las filas usadas (clave con $V \approx 10^5$; requiere SGD/SparseAdam/Adagrad). `nn.Embedding.from_pretrained(M, freeze)` crea la tabla desde una matriz (GloVe, SGNS); `freeze=True` la congela (`requires_grad=False`) y `freeze=False` permite *fine-tuning*.
- `nn.EmbeddingBag(..., mode)` agrega bolsas de vectores sin materializar el tensor (B, L, d) : recibe todos los IDs concatenados y `offsets` (inicio de cada documento), así que no necesita padding. `mean` promedia (invariante a la longitud), `sum` suma (admite `per_sample_weights`) y `max` toma el máximo por dimensión.
- **gensim**: `Word2Vec(vector_size, window, min_count, sg, negative, sample, alpha, epochs)`; `KeyedVectors` ofrece `most_similar` (3CosAdd), `most_similar_cosmul`, `evaluate_word_analogies`, `evaluate_word_pairs` y `save_word2vec_format`.
- **CBOW vs. skip-gram**: CBOW predice la palabra central desde el promedio del contexto (rápido, favorece palabras frecuentes); skip-gram predice cada contexto desde la central (más pares por token, mejor con palabras raras y analogías).
- **Dos matrices**: W (palabra como centro) y W' (palabra como contexto/negativo). Con una sola matriz el modelo debería dar baja probabilidad a $\sigma(v_w^\top v_w)$, que es siempre grande, y la optimización se degrada. Se conserva W .
- **Word2Vec vs. GloVe**: Word2Vec es *predictivo* (SGD local sobre ventanas, contexto real vs. negativos); GloVe es *de conteo*: construye la matriz global X_{ij} y ajusta $w_i^\top \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$ por mínimos cuadrados ponderados. SGNS factoriza implícitamente una PMI desplazada (Levy & Goldberg, 2014).

2. Exploración y preprocesamiento

Corpus. El split `train` de WikiText-103 tiene 1,801,350 líneas (1,165,029 no vacías, de las cuales 275,630 son encabezados de sección), 29,444 líneas de título de artículo (el dataset reporta 28,475 artículos; el resto son líneas con el mismo formato " = X = ") y 101,425,671 tokens separados por espacios (84,593,278 tras nuestra normalización, que elimina la puntuación; 717,430 tipos).

Normalización (alineada con GloVe 6B, *uncased* y tokenizado estilo PTB). (i) minúsculas; (ii) se reconstruyen los artefactos de WikiText (`well @-@ known→well-known`, `1 @,@ 000→1,000`, `1 @.@ 5→1.5`); (iii) contracciones PTB (`didn't→did n't`, `can't→ca n't`, `'s` separado); (iv) compuestos con guion y siglas/abreviaturas con punto (`u.s.`, `dr.`, `jan.`) como un solo token; (v) números como dígitos (sin <num>); (vi) puntuación eliminada (no aparece en los benchmarks y ocuparía posiciones de la ventana). En los embeddings las palabras bajo `min_count` se *eliminan* antes de formar ventanas (como Word2Vec); en clasificación no se necesita <pad> porque `EmbeddingBag` usa `offsets`. Lo que *debe* coincidir con GloVe es el casing, el formato de contracciones, guiones y números: si no, una misma palabra tendría formas distintas y los benchmarks se evaluarían sobre preguntas diferentes.

Tokenizador propio vs. librerías (12 oraciones, sin puntuación). Coincide con NLTK salvo en siglas finales (`u.s.` vs. `u.s,e.g.`), `i'd've` (NLTK deja `i'd`) y correos. `spaCy` difiere más: parte los compuestos con guion (`state | of | the | art,york | based, 10 | 15`), quita el punto de `dr./jan.` y conserva `jsmith@example.com`. Las tres separan igual `ca n't`, `wo n't` e `is n't`.



<code>min_count</code>	$ V $	% desc.	palabra	$t=10^{-3}$	$t=10^{-4}$	$t=10^{-5}$
1	392,765	0.00	the	87.3	96.3	98.8
3	153,008	0.98	of	79.5	94.2	98.2
5	109,700	1.47	and	78.2	93.8	98.1
10	72,652	2.28	corpus	22.6	39.9	64.1
20	48,370	3.38	(% de apariciones descartadas)			
50	27,810	5.51				

Figura 1: Izq.: ley de Zipf en WikiText-103 (85 M tokens). Der.: vocabulario y % de tokens desconocidos por `min_count` en el subconjunto de 30 M tokens, y % de apariciones descartadas por el submuestreo.

Zipf, vocabulario y submuestreo. La curva rango-frecuencia es casi una recta en log-log con exponente $s = 1.06$ (ajuste en rangos 10–10,000): se cumple la ley de Zipf, con desviación en la cola (hápx). Las 10, 1,000 y 30,000 palabras más frecuentes cubren 24.6 %, 64.0 % y 94.6 % de los tokens. Elegimos `min_count=5`: $|V| = 109,700$ con sólo 1.5 % de tokens perdidos (la mayoría, nombres raros y errores). Con la fórmula de Word2Vec $P_{\text{keep}} = (\sqrt{f/t} + 1)t/f$ y $t = 10^{-4}$ se descarta el 96 % de `the` y el 94 % de `of/and` (40 % del corpus). Ayuda porque esas palabras aportan poca información sobre sus vecinas, reduce los pares a la mitad y, al quitarlas antes de formar ventanas, éstas abarcan más palabras de contenido. **Pares skip-gram** (ventana 5, sin cruzar párrafos): 286.6 M por epoch con ventana fija y 173.1 M con ventana dinámica ($b \sim U\{1..5\}$) antes del submuestreo; 168.8 M y **102.4 M** después (17.8 M tokens conservados).

3. Construcción y entrenamiento de los embeddings

SGNS propio. Dos tablas `nn.Embedding(sparse=True)` ($W \sim U(\pm 0.5/d)$, $W' = 0$), pérdida $-\log \sigma(u_o^\top v_c) - \sum_k \log \sigma(-u_{n_k}^\top v_c)$ con negativos de $P_n \propto \text{count}^{0.75}$, `SparseAdam` con batch 8,192 y decaimiento lineal del lr. Como Word2Vec: submuestreo nuevo en cada epoch, ventana dinámica y ventanas sin cruzar párrafos; los pares se generan en GPU por bloques de 2M tokens y se barajan. **gensim**: mismo archivo de corpus, mismo `min_count` (vocabulario idéntico) y mismos `window`, `negative`, `sample`, `vector_size`, `epochs`; el lr no es equivalente (SGD $0.025 \rightarrow 10^{-4}$), así que se dejó el de gensim. **Selección** sólo con WordSim-353 y analogías (top-30,000 de cada modelo, como `restrict_vocab`): $\text{score} = (\text{acc}_{\text{analogías}} + \rho_{\text{ws}})/2$. El evaluador propio en GPU reproduce exactamente a gensim (GloVe: 0.6549 y 0.5327). Primero se barrió el lr y luego se varió un factor a la vez; la combinación ganadora sólo cambió los negativos ($K = 15$) y se reentrenó con 10 epochs.

Iteración	tokens	$ V $	d	win	K	t	lr	ep	pérdida	sem	sin	total	WS-353	s/epoch
lr 2×10^{-3}	30.0 M	109.7k	100	5	5	10^{-4}	0.002	5	2.173	.283	.452	.401	.665	43
base (lr 5×10^{-3})	30.0 M	109.7k	100	5	5	10^{-4}	0.005	5	2.140	.387	.471	.446	.674	52
lr 10^{-2}	30.0 M	109.7k	100	5	5	10^{-4}	0.01	5	2.136	.404	.468	.449	.666	62
$d = 50$	30.0 M	109.7k	50	5	5	10^{-4}	0.005	5	2.189	.251	.390	.349	.635	47
$d = 300$	30.0 M	109.7k	300	5	5	10^{-4}	0.005	5	2.057	.523	.483	.495	.663	199
ventana 2	30.0 M	109.7k	100	2	5	10^{-4}	0.005	5	2.008	.328	.483	.437	.656	106
ventana 10	30.0 M	109.7k	100	10	5	10^{-4}	0.005	5	2.205	.418	.461	.448	.668	242
$K = 2$	30.0 M	109.7k	100	5	2	10^{-4}	0.005	5	1.491	.342	.447	.416	.684	181
$K = 15$	30.0 M	109.7k	100	5	15	10^{-4}	0.005	5	3.046	.448	.486	.475	.671	293
$t = 10^{-3}$	30.0 M	109.7k	100	5	5	10^{-3}	0.005	5	2.184	.376	.494	.458	.655	280
$t = 10^{-5}$	30.0 M	109.7k	100	5	5	10^{-5}	0.005	5	2.047	.394	.399	.397	.668	125
<code>min_count=10</code>	30.0 M	72.7k	100	5	5	10^{-4}	0.005	5	2.170	.377	.474	.445	.674	104
corpus 25 %	7.5 M	50.3k	100	5	5	10^{-4}	0.005	5	2.133	.172	.247	.224	.618	48
corpus 50 %	15.0 M	74.7k	100	5	5	10^{-4}	0.005	5	2.134	.324	.396	.374	.651	86
mejor ($K=15$, 10 ep)	30.0 M	109.7k	100	5	15	10^{-4}	0.005	10	3.028	.474	.496	.490	.674	277
gensim (mejor cfg)	30.0 M	109.7k	100	5	15	10^{-4}	SGD	10	—	.461	.502	.490	.669	101 [†]

Tabla 1: Iteraciones (métricas al final del entrenamiento; analogías sobre las 30,000 palabras más frecuentes de cada modelo). Memoria GPU pico: 2.1–3.5 GB (3.1 GB con $d = 300$). [†]CPU, 8 hilos. Los tiempos están inflados y son ruidosos: la GPU (RTX 3070 Laptop) entró en *throttling* térmico y compartió recursos con otros procesos (las primeras epochs tardaban ≈ 36 s). Tiempo total del mejor SGNS: 46 min; gensim: 17 min.

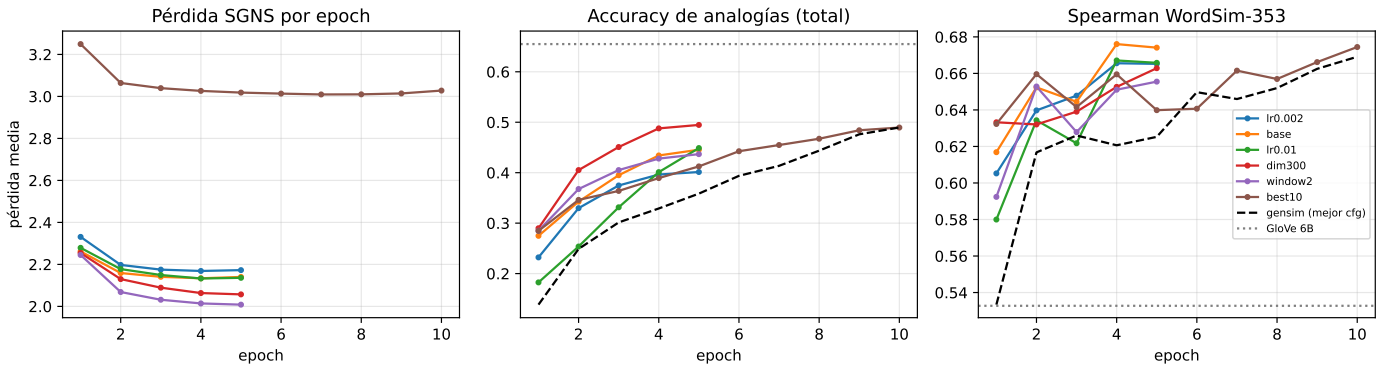


Figura 2: Pérdida, accuracy de analogías y Spearman en WordSim-353 por epoch para 6 iteraciones y gensim (línea negra).

¿Menor pérdida $\Rightarrow$ mejores embeddings? No. La pérdida depende de la tarea: $K = 2$ tiene la menor pérdida (1.49) y la ventana 2 la segunda (2.01), y ambas dan peores analogías que la base; $K = 15$ tiene pérdida 3.05 y es la mejor. Incluso dentro de una corrida la pérdida del mejor modelo *sube* de 3.010 (epoch 8) a 3.028 (epoch 10) mientras las analogías mejoran de 46.7 a 48.9 %. Los vecinos de control se vuelven coherentes desde la epoch 2 (`france` $\rightarrow$ `spain`, `italy`, `belgium`; `computer` $\rightarrow$ `computers`, `software`, `computing`; `january` $\rightarrow$ otros meses); `king` queda dominado por nombres de reyes raros (`osric`, `wenceslaus`), un efecto del corpus enciclopédico que gensim también muestra.

4. Verificación de la aritmética vectorial

Analogías individuales. $\text{analogia}(a,b,c,k)$ usa 3CosAdd, $\arg \max_x \cos(x, \hat{b} - \hat{a} + \hat{c})$, sobre vectores normalizados (top-100,000) excluyendo a, b, c . Devuelve las mismas palabras y cosenos que `most_similar` de gensim en los tres modelos. Se probaron 20 analogías propias de 7 tipos. Aciertos top-1: SGNS 15/20, gensim 14/20, GloVe 17/20. Todos aciertan gentilicios, tiempos verbales (`go:went::take:took`), comparativos y plurales regulares. Los fallos más ilustrativos (rango de la respuesta): `man:king::woman:queen` (SGNS 2, gensim 5, GloVe 1; SGNS propone `jagiellon` con 0.74 vs. `queen` 0.73); `canada:ottawa::australia:canberra` (250 / 120 / 2: Canberra aparece poco en 30 M tokens); `mouse:mice::foot:feet` (76 / 36 / 1); `big:biggest::small:smallest` (11 / 6 / 11; todos proponen `large/largest`) y `possible:impossible::legal:illegal` (589 / 165 / 268). El coseno entre el vector resultado y la respuesta correcta nunca es 1: 0.73 (SGNS), 0.69 (gensim) y 0.77 (GloVe) para `queen`.

Sin excluir la consulta, la primera palabra es b o c en 12/20 (SGNS), 13/20 (gensim) y 9/20 (GloVe) analogías. Para `man:king::woman:?` los tres devuelven `king` (0.77, 0.78, 0.84) antes que `queen`; para `walk:walked::swim:?` SGNS y gensim devuelven `swim`. Es decir, $\hat{b} - \hat{a} + \hat{c}$ suele quedar más cerca de una palabra de la consulta que de la respuesta: la "aritmética" sólo funciona como búsqueda del vecino más cercano *después* de excluir esas palabras.

Categoría	cobert.	SGNS propio		gensim		GloVe		paralelismo (cos medio)		
		Add	Mul	Add	Mul	Add	Mul	SGNS	gensim	GloVe
capital-common-countries	.75	.708	.668	.729	.668	.929	.911	.34	.35	.58
capital-world	.23	.571	.523	.579	.521	.883	.859	.25	.26	.57
currency	.06	.019	.019	.019	.019	.056	.056	.28	.24	.25
city-in-state	.71	.361	.326	.324	.299	.302	.268	.18	.19	.34
family	.68	.564	.503	.576	.529	.880	.868	.25	.25	.51
adjective-to-adverb	.82	.121	.105	.132	.113	.280	.278	.12	.12	.27
opposite	.34	.070	.063	.143	.103	.272	.239	.09	.10	.23
comparative	1.00	.517	.503	.578	.552	.800	.781	.30	.31	.42
superlative	.63	.325	.313	.298	.295	.608	.611	.28	.28	.47
present-participle	.82	.438	.386	.430	.386	.722	.716	.16	.18	.32
nationality-adjective	.81	.865	.851	.851	.826	.974	.958	.47	.48	.75
past-tense	.85	.630	.597	.619	.559	.588	.562	.20	.20	.29
plural	.74	.510	.457	.460	.408	.799	.764	.16	.16	.31
plural-verbs	.75	.335	.348	.383	.385	.583	.611	.26	.26	.36
Semánticas	.40	.474	.433	.461	.421	.590	.564			
Sintácticas	.77	.496	.473	.500	.469	.683	.670			
Total	.61	.490	.461	.489	.454	.655	.638	.24 (.12)	.24 (.12)	.40 (.17)

Tabla 2: Accuracy en `questions-words.txt` sobre el vocabulario compartido (las 30,000 palabras más frecuentes de nuestro corpus presentes en los tres modelos; 11,834 de 19,544 preguntas, 60.6 %). Paralelismo: coseno medio entre todos los pares de vectores diferencia $\hat{b} - \hat{a}$ de la categoría; entre paréntesis, la línea base con pares mal emparejados.

Evaluación sistemática y paralelismo. Con las mismas 11,834 preguntas, SGNS y gensim empatan (48.9 %) y GloVe llega a 65.5 %. 3CosMul no mejoró a 3CosAdd en ningún modelo (-2 a -4 puntos) con este vocabulario restringido. Si la aritmética fuera exacta, los vectores diferencia de una relación serían paralelos (coseno 1); en SGNS el coseno medio es sólo 0.24 (0.40 en GloVe), aunque el doble de la línea base aleatoria. Las relaciones más paralelas (gentilicios 0.47, países-capitales 0.34) son justo las de mayor accuracy, y las menos paralelas (opuestos 0.09, adjetivo-adverbio 0.12) las peores. **t-SNE** de 479 palabras en 14 grupos temáticos (figura en el notebook): se forman los grupos esperados. La pureza de los 5 vecinos más cercanos en el espacio original es 0.84 en SGNS y 0.89 en GloVe: meses/días e instrumentos son perfectos (1.00), números y deportes casi (0.97), y los más mezclados son capitales (0.62, se confunden con países y ciudades de EE. UU.) y profesiones (0.74).

5. Pipeline end-to-end: clasificación de AG News

Mismo `tokenize` de la sección 2; 10 % estratificado de train como validación (108,000/12,000) y test oficial (7,600) usado una sola vez. Clasificador: `EmbeddingBag(mean)` $\rightarrow$ `Linear(100,256)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.3)` $\rightarrow$ `Linear(256,4)`; Adam (2×10^{-3}), batch 256, hasta 8 epochs con *early stopping* por F1 de validación. Vocabulario aleatorio: palabras de train con frecuencia ≥ 2 ; preentrenado: palabras del modelo presentes en AG News (las fuera de vocabulario se descartan). Tokens de AG News (test) fuera de vocabulario: **SGNS y gensim 3.3 %, GloVe 0.8 %**. Baseline: TF-IDF de uni+bigramas (`min_df=2`, `sublinear_tf`) + regresión logística ($C = 4$ elegido en validación).

Modelo	Validación (macro)				params totales	entrenables	tiempo	Test (mejor variante)	
	Acc	Prec	Rec	F1				Acc	F1
TF-IDF + LogReg	.9248	.9246	.9248	.9246	1,698,048	1,698,048	151 s	.9213	.9211
Aleatorio (entrenado)	.9143	.9145	.9143	.9142	4,950,084	4,950,084	6 s	.9055	.9055
SGNS congelado	.8856	.8856	.8856	.8854	4,566,384	26,884	4 s		
SGNS fine-tuning	.9188	.9187	.9188	.9186	4,566,384	4,566,384	4 s	.9141	.9139
gensim congelado	.8868	.8893	.8868	.8867	4,566,384	26,884	4 s		
gensim fine-tuning	.9185	.9195	.9185	.9185	4,566,384	4,566,384	5 s	.9136	.9136
GloVe congelado	.9036	.9041	.9036	.9036	6,628,384	26,884	4 s		
GloVe fine-tuning	.9229	.9236	.9229	.9229	6,628,384	6,628,384	6 s	.9183	.9183

Tabla 3: Clasificación de AG News (las curvas de pérdida de entrenamiento y validación están en el notebook).

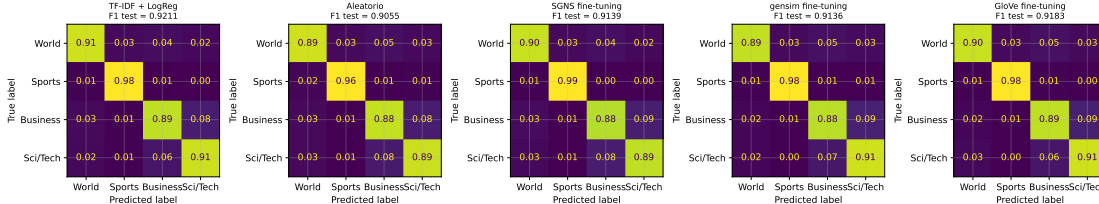


Figura 3: Matrices de confusión normalizadas en test de la mejor variante de cada inicialización. En todos los modelos la mayor confusión es Business↔Sci/Tech (6–9 %) y la clase más fácil es Sports (96–99 %).

SimLex-999 (reservado hasta el final): $\rho = 0.295$ (SGNS), 0.312 (gensim) y 0.298 (GloVe), con cobertura de 99.6–100 %. A diferencia de WordSim-353 (que mide *relación*, donde nuestros modelos superan a GloVe: 0.674 vs. 0.533), SimLex mide *similitud* estricta y penaliza que antónimos y palabras relacionadas compartan contexto: los tres modelos quedan igual de mal.

6. Comparación de resultados

	SGNS propio (PyTorch)	Word2Vec (gensim)	GloVe 6B (preentrenado)
Tokens / vocabulario / dimensión	30.0 M / 109,700 / 100	30.0 M / 109,700 / 100	6 B / 400,000 / 100
Tiempo y hardware	46 min, RTX 3070 Laptop (8 GB)	17 min, CPU 8 hilos	≈85 min co-ocurrencias + 50 iter. (2× Xeon E5-2658, 32 núcleos)
Analogías 3CosAdd (sem / sin / total)	.474 / .496 / .490	.461 / .500 / .489	.590 / .683 / .655
Analogías 3CosMul (sem / sin / total)	.433 / .473 / .461	.421 / .469 / .454	.564 / .670 / .638
Spearman WordSim-353 / SimLex-999	.674 / .295	.669 / .312	.533 / .298
Cos. medio vectores diferencia (base)	.238 (.116)	.241 (.118)	.404 (.175)
OOV en AG News (test) / F1 test	3.3 % / .9139	3.3 % / .9136	0.8 % / .9183

Tabla 4: Comparación de los tres conjuntos de embeddings. Tiempos de GloVe según Pennington et al. (2014).

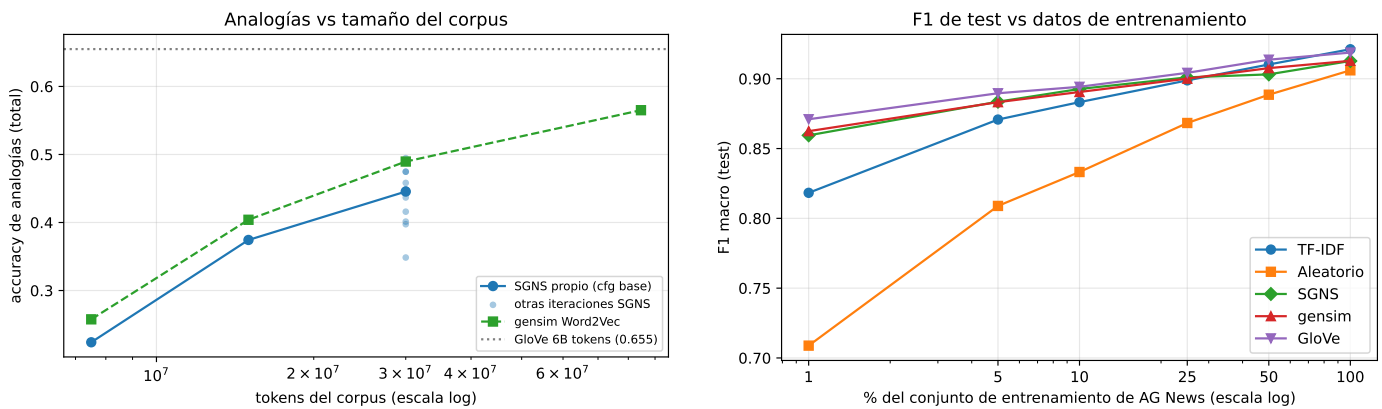


Figura 4: Izq.: accuracy de analogías vs. tokens del corpus (escala log) para nuestras iteraciones y gensim (incluye el split completo de WikiText-103, 85 M tokens), con GloVe como referencia. Der.: F1 de test vs. fracción de AG News usada para entrenar; para cada inicialización se usa la variante (congelada/fine-tuning) elegida en validación; 1 % y 5 % promedian 3 semillas.

7. Discusión y conclusiones

Aritmética vectorial. Se cumple *aproximadamente*: el mejor SGNS resuelve el 49 % de las analogías y 15 de nuestras 20. Funciona mejor en relaciones morfológicas regulares o muy frecuentes: gentilicios (86 %), países-capitales comunes (71 %) y pasado (63 %). Falla en monedas (2 %, y sólo 6 % de cobertura), opuestos (7 %) y adjetivo-adverbio (12 %). Las semánticas (47 %) quedan un poco por debajo de las sintácticas (50 %): las relaciones semánticas dependen de hechos

(qué capital va con qué país) que necesitan ver muchas veces cada par, mientras que la morfología se aprende con pocos ejemplos frecuentes.

¿Es « $\approx$ » una igualdad? No. Sin excluir la consulta, *b* o *c* ganan en la mayoría de los casos (*king* queda antes que *queen* en los tres modelos), el coseno con la respuesta correcta es 0.7–0.8 y los vectores diferencia de una misma relación tienen coseno medio 0.24 (0.40 en GloVe), lejos del 1 que implicaría una igualdad: es un desplazamiento promedio ruidoso, no una identidad.

Distancia a GloVe. 16.6 puntos en analogías. El experimento de corpus indica que la mayor parte se debe a los datos. Con el mismo método y los mismos hiperparámetros, gensim pasa de 25.8 % (7.5 M) a 40.4 % (15 M), 49.0 % (30 M) y 56.5 % (85 M). Sólo pasar de 30 M a 85 M cierra 7.5 de los 16.6 puntos (45 %), y GloVe tiene 70 veces más tokens que eso, aunque las ganancias por duplicación decrecen (+15, +9 y +5 puntos por duplicación aprox.). El resto se atribuye al método y a los hiperparámetros: $d = 300$ suma 5 puntos, GloVe usa ventana 10, 50 iteraciones y estadísticas globales, y su vocabulario de 400 k cubre las palabras raras de las analogías (capital-world: 23 % de cobertura en el vocabulario compartido).

SGNS propio vs. gensim. Se comportan casi igual: 48.95 % vs. 48.96 % total, ρ_{WS} 0.674 vs. 0.669, paralelismo 0.238 vs. 0.241 y F1 en AG News 0.9139 vs. 0.9136, con diferencias por categoría de ± 5 puntos y los mismos vecinos atípicos (*jagiellon* para *queen*). Las diferencias vienen del optimizador: **SparseAdam** con batches de 8,192 pares frente al SGD asíncrono por par de gensim. Por eso el nuestro mejora más rápido en las primeras epochs (Fig. 2), mientras gensim alcanza el mismo punto al final cuando su lr decae a cero. También influyen el barajado por bloques y la semilla. La pérdida de gensim no es comparable (se acumula en float32).

Hiperparámetros. *Dimensión:* 50→100→300 da 34.9→44.5→49.5 %, sobre todo en semánticas (25→39→52 %), mientras que WordSim se satura en $d = 100$. *Ventana:* 2→5→10 sube las semánticas (32.8→38.7→41.8 %) y baja las sintácticas (48.3→47.1→46.1 %): **las ventanas pequeñas favorecen las relaciones sintácticas** (contexto inmediato: morfología y función) y las grandes, las temáticas o semánticas. *Negativos:* 2→5→15 da 41.6→44.5→47.5 % en analogías, aunque $K = 2$ obtuvo el mejor WordSim. Con $t = 10^{-5}$ se descartan demasiadas palabras funcionales y caen las sintácticas (39.9 %). Con `min_count=10` el resultado es igual con un vocabulario 34 % menor. El lr óptimo de **SparseAdam** fue 5×10^{-3} , y duplicar las epochs sumó 1.5 puntos.

AG News. Los preentrenados con fine-tuning superaron a los aleatorios (F1 test 0.914/0.914/0.918 vs. 0.906), pero *no* al baseline TF-IDF con bigramas (0.921): con 108,000 ejemplos y clases separables por palabras clave, un modelo lineal disperso es muy fuerte, y el promedio de embeddings pierde el orden y los bigramas. Con todos los datos conviene el fine-tuning: congelar cuesta 3–4 puntos, porque la tabla no puede adaptarse a la tarea, aunque entrena 170 veces menos parámetros. **Con 1 % de los datos (1,080 ejemplos) la respuesta cambia:** los preentrenados (SGNS 0.860, GloVe 0.871) superan por 15–16 puntos a los aleatorios (0.709) y por 4–5 a TF-IDF (0.818), y congelar rinde casi lo mismo que el fine-tuning (GloVe 0.868 vs. 0.871, SGNS 0.854 vs. 0.860) con mucho menos riesgo de sobreajuste. Los preentrenados sirven sobre todo para aprovechar datos no etiquetados cuando hay pocas etiquetas. GloVe gana sobre SGNS principalmente por su menor OOV (0.8 % vs. 3.3 %) y por haber visto noticias (Gigaword).

Repositorio: <https://github.com/anthonylouschwank/Lab7-DL>